Residência Tecnológica — UFPE/CIn

# aneb-sim

Simulador da placa ANEB v1.1

Sistemas Operacionais de Tempo Real — Manual do Usuário

| | |
|---|---|
| **Versão** | 0.9 (M9 / pré-release) |
| **Data** | Maio de 2026 |
| **Repositório** | `renatosfagundes/aneb-sim` |
| **Plataforma** | Windows 10/11 (MSYS2 + Python 3.11) |

*Gerado por `docs/manual/manual.tex` — Compilado com Tectonic*

# Sumário

**1 Introdução** **3**
1.1 Por que um simulador? . . . 3
1.2 Funcionalidades . . . 3

**2 Instalação** **4**
2.1 Pré-requisitos . . . 4
2.2 Clone e bootstrap . . . 4
2.3 Primeira compilação . . . 4
2.4 Configuração das portas COM virtuais (com0com) . . . 4

**3 Primeira execução** **5**
3.1 O que acontece ao iniciar . . . 6

**4 Visão geral da interface** **6**
4.1 Painel ECU . . . 6
4.2 Painel MCU . . . 6
4.3 Toolbar . . . 7

**5 Console serial** **7**
5.1 Cabeçalho . . . 7
5.2 Limites de buffer . . . 8
5.3 Envio para o chip . . . 8

**6 Plotter** **8**
6.1 Cabeçalho . . . 9
6.2 Catálogo de sinais . . . 9
6.3 Amostragem . . . 9

**7 Gravação de firmware** **10**
7.1 Pelo botão Flash da UI . . . 10
7.2 Externamente (Remote Flasher, Arduino IDE, linha de comando) . . . 10

**8 Integração com o Remote Firmware Flasher** **11**
8.1 Fluxo . . . 11

**9 Exemplos de firmware** **11**
9.1 Catálogo . . . 12
9.2 Compilando . . . 12
9.3 Fluxo recomendado de aprendizado . . . 12

**10 Cenários: *teaching aids* reproduzíveis** **13**
10.1 Formato . . . 13
10.2 Catálogo . . . 13
10.3 Como executar . . . 13
10.4 Criando seus próprios cenários . . . 14

**11 Controle programático: protocolo JSON-Lines** **14**

**12 Solução de problemas** **14**
12.1 A UI abre mas não vejo dados nas portas COM . . . . . 14
12.2 Avrdude trava em “not in sync” . . . . . 15
12.3 O simulador trava ou “Not Responding” . . . . . 15
12.4 Quero recompilar Optiboot ou usar outro bootloader . . . . . 15

**Apêndice A — Mapa de portas TCP** **15**

**Apêndice B — Estrutura de diretórios** **16**

# 1. Introdução

O **aneb-sim** é um simulador de instruções da placa **Automotive Network Evaluation Board v1.1** (ANEB) usada na disciplina de Sistemas Operacionais de Tempo Real do CIn/UFPE. Reproduz, em um único processo no PC do aluno, o comportamento dos cinco microcontroladores da placa real:

- **ECU1-ECU4**: quatro ATmega328P, cada uma ligada a um controlador CAN MCP2515, dois LEDs digitais (DOUT0/DOUT1), quatro entradas analógicas com potenciômetros virtuais (AIN0–AIN3), quatro botões de entrada digital (DIN1–DIN4), um buzzer e um LCD $I^2C$ 16×2;
- **MCU**: um ATmega328PB que roda o firmware de controle da placa (`MCU_Firmware_v08.hex`) e expõe os seletores de modo;
- **Barramento CAN1**: rede compartilhada entre as quatro ECUs, modelada com fidelidade de bit-stuffing, contadores TEC/REC e estados *active* / *passive* / *bus-off*;

## 1.1 Por que um simulador?

- **Acessibilidade**: não exige a placa física nem conexão VPN ao laboratório.
- **Reprodutibilidade**: o mesmo `.hex` compilado para a placa real roda no simulador — não há porte de código.
- **Ferramentas**: o aluno usa `avrdude`, monitores seriais, e até o *Remote Firmware Flasher* apontados para o simulador exatamente como apontariam para uma placa.
- **Diagnóstico**: estados internos (TEC/REC, free-RAM, sinais de barramento) ficam visíveis em tempo real, o que é difícil de obter em hardware.

## 1.2 Funcionalidades

- Engine em C com `simavr` (cinco núcleos AVR em paralelo, passo de *lockstep* determinístico);
- Modelo MCP2515 fiel a nível de registrador (filtros, EFLG, bus-off, recuperação);
- Interface PyQt6/QML com painel por chip — ilustração do Arduino Nano, LCD, LEDs domeados, potenciômetros angulares, botões DIN coloridos;
- Console serial por chip com texto colorido, contador de bytes, LED de atividade RX e *follow*-toggle;
- Plotter por chip com até 14 sinais simultâneos (ADC, PWM, digitais), janela temporal selecionável e *pause/resume*;
- Painel de gravação `avrdude` com log colorizado e indicador de status (*idle/-flashing/success/done*);
- **Optiboot embutido**: todo chip nasce com bootloader pré-carregado, então qualquer ferramenta (avrdude, Arduino IDE, Remote Flasher) consegue gravar firmware sem passos manuais;
- **Servidor TCP UART duplo**: porta de *flasher* (`8600+i`) para gravação STK500 + porta de *bridge* (`8700+i`) para encaminhamento passivo via portas COM virtuais (com0com);

- Integração com o **Remote Firmware Flasher** v2.4.0+ como entrada `Localhost (aneb-sim)`.

# 2. Instalação

## 2.1 Pré-requisitos

1. **Windows 10 ou 11** (64 bits).
2. **MSYS2** com a toolchain `mingw-w64` — veja `BOOTSTRAP.md` no repositório para passos detalhados.
3. **Python 3.11.x** de https://www.python.org/downloads/release/python-3112/ (não use 3.12 ou superior; o pacote `PyQt6` usado pela UI ainda fixa 3.11 nas suas roads).
4. **Git** (qualquer versão recente).

## 2.2 Clone e bootstrap

```
git clone --recurse-submodules \
    https://github.com/renatosfagundes/aneb-sim.git /c/dev/aneb-sim
cd /c/dev/aneb-sim
./scripts/bootstrap.sh
```

O script `bootstrap.sh`:

- instala dependências do MSYS2 (`ninja`, `cmake`, `libelf-dev`, etc.);
- compila o submódulo `simavr`;
- cria o `venv` Python em `.venv/` e instala `PyQt6`, `pyserial` e similares.

## 2.3 Primeira compilação

```
./scripts/build.sh
```

O *output* é `build/aneb-sim/aneb-sim.exe`. Esse é o *engine* (binário C); a UI é Python e roda do mesmo `venv`.

## 2.4 Configuração das portas COM virtuais (com0com)

Para que ferramentas externas (Remote Flasher, monitores seriais, `avrdude` pela linha de comando) enxerguem cada chip do simulador como uma porta COM, é necessário instalar pares `com0com` virtuais.

```
# Como administrador (PowerShell elevado ou cmd "Run as Admin")
scripts\setup_com.bat
```

O script:

1. instala o driver assinado da redistribuição do *Pete Batard* (em `scripts\com0com_signed/`);
2. cria cinco pares de portas COM virtuais e os marca no registro do Windows com `FriendlyName ECU1 (aneb-sim)`, `ECU2 (aneb-sim)`, , `MCU (aneb-sim)`;

3. escolhe automaticamente nomes COM livres (tipicamente `COM10/COM11`, `COM12/COM13`, ).

A UI do aneb-sim e o Remote Flasher leem o registro para descobrir qual porta COM corresponde a cada chip — não há configuração manual.

**Removendo as portas**

Para desinstalar os pares e o driver:

```
scripts\remove_com.bat
```

Pode ser útil ao trocar de máquina, atualizar o Windows ou liberar números COM para outros usos.

# 3. Primeira execução

```
./scripts/run-ui.sh
```

A janela principal abre com o layout completo da placa: as quatro ECUs em *grid* 2×2 à esquerda e o painel MCU à direita. A barra superior mostra o título **ANEB v1.1** junto a botões de controle (**Load**, **Pause**, **Resume**, **Speed**, **1×**, **Max**, **Dashboard**) e ao indicador de *engine* (*running* em verde).

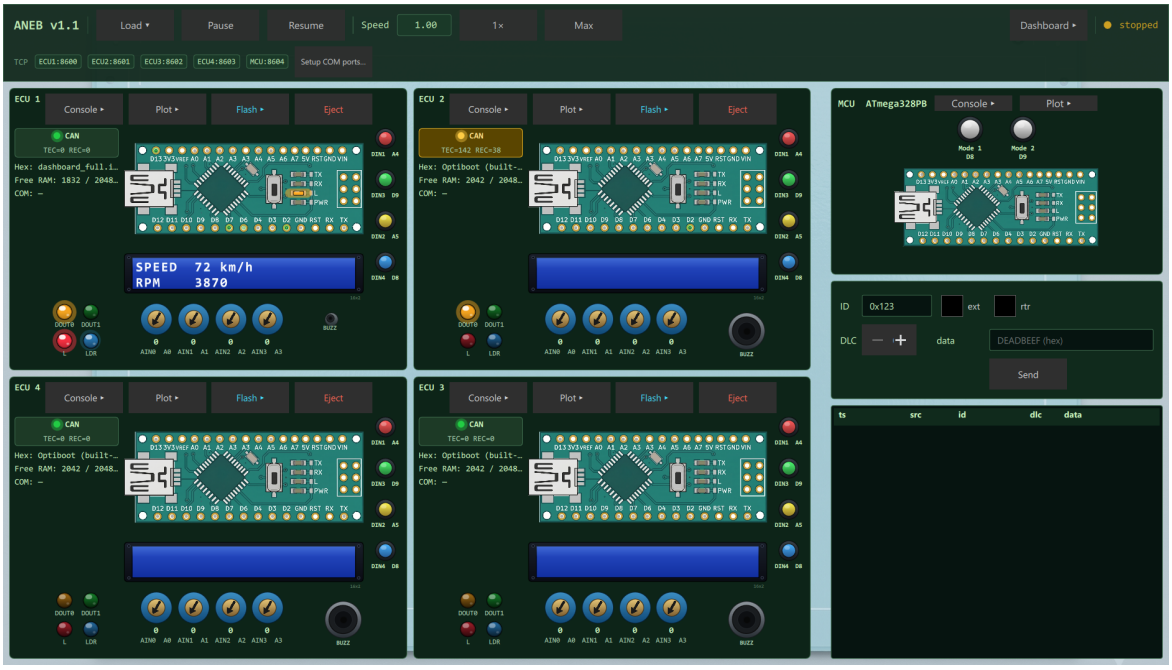


Figura 1: Janela principal do aneb-sim com as quatro ECUs e o MCU. Cada *card* mostra a barra lateral com o status CAN e os metadados do firmware (Hex, Free RAM, COM), a ilustração do Arduino Nano e o LCD.

## 3.1 O que acontece ao iniciar

1. O *engine* (`aneb-sim.exe`) é lançado como subprocesso da UI; sua *stdout* JSON-Lines alimenta as bindings QML, e a *stdin* recebe comandos da UI.
2. Cada chip ATmega328P recebe automaticamente o **Optiboot** pré-carregado em *flash* — nasce pronto para receber comandos STK500.
3. A *bridge* interna conecta as cinco portas TCP UART (`8700+i`) às portas COM virtuais (`COM10`–`COM18`, lado bridge) e expõe a contraparte (`COM11`–`COM19`) para o usuário.
4. Sem firmware do usuário, cada ECU executa um *stub* `rjmp .self` — ou seja, fica em *idle* sem gerar tráfego. As janelas de Console, Plotter e Flash permanecem fechadas.

# 4. Visão geral da interface

## 4.1 Painel ECU

Cada uma das quatro ECUs ocupa um *card* com a seguinte disposição:

- **Linha de título** (topo): rótulo (`ECU 1`, `ECU 2`, ) e três botões para abrir/fechar as janelas auxiliares: **Console**, **Plot** e **Flash**, mais um botão **Eject** para descarregar o firmware.
- **Barra lateral** (esquerda): pílula CAN com indicador de estado e contadores TEC/REC; abaixo dela, três linhas de metadados:
  - `Hex:`, com o nome do firmware atualmente em *flash* (`Optiboot (built-in)` ao iniciar, ou o nome do `.hex` carregado);
  - `Free RAM:`, com o valor estimado de SRAM livre (em bytes / total) calculado a partir do SP atual do chip;
  - `COM:`, com a porta COM virtual da *bridge* que o usuário deve abrir para ler/escrever a UART.

  Passar o mouse sobre qualquer linha mostra um *tooltip* com a explicação detalhada do valor.
- **Centro**: ilustração SVG do Arduino Nano com pinagem rotulada (`D0`–`D13`, `A0`–`A7`) e LEDs internos (`TX`, `RX`, `L`, `PWR`). Logo abaixo, o LCD $I^2C$ 16×2.
- **Botões DIN** (direita): quatro *push-buttons* coloridos — vermelho (DIN1/A4), verde (DIN3/D9), amarelo (DIN2/A5) e azul (DIN4/D8). Clicar simula o pressionamento físico do botão na placa real.
- **LEDs e potenciômetros** (rodapé): `DOUT0` e `DOUT1` (LEDs digitais), `L` (LED na placa), `LDR` (LED associado ao laço óptico), os quatro *trim-pots* angulares para `AIN0`–`AIN3` e o `BUZZ` (representação do buzzer).

## 4.2 Painel MCU

À direita, o painel **MCU** mostra o ATmega328PB. Como o MCU não está conectado ao barramento CAN nem aos periféricos das ECUs, seu *card* é mais enxuto: ilustração do Nano, LCD, e dois seletores de *modo* (`Mode 1` D8 / `Mode 2` D9).

### 4.3 Toolbar

- **Load** : menu de carregamento rápido que navega para `firmware/examples/` ou `firmware/mcu/` dependendo do contexto;
- **Pause/Resume**: pausam/retomam todos os chips de uma vez (útil para inspecionar estado);
- **Speed** (`0.5×` / `1×` / `Max`): multiplicador de *wallclock*. `1×` é tempo real (16 MHz), `Max` libera a simulação para correr o mais rápido que o PC permitir (útil para acelerar testes longos);
- **Dashboard**: abre uma janela compacta com a leitura ao vivo de todas as ECUs.

## 5. Console serial

Cada ECU e o MCU têm uma janela de console serial dedicada, acessível pelo botão **Console >** no painel do chip. A janela mostra a saída UART do firmware em execução e permite enviar caracteres de volta ao chip.

Figura 2: Janela de console serial mostrando saída UART de um sketch Plotter Arduino. Cabeçalho com chip badge, contador de bytes, LED de atividade RX, botão *Clear* e *toggle Follow*; rodapé com campo de envio.

### 5.1 Cabeçalho

Da esquerda para a direita: emblema do chip (`ECU 1`), ponto acentuado em ciano com o título **Console**, contador de bytes recebidos, ponto verde que pisca a cada chunk de UART e três controles de canto: **Clear** (apaga o buffer visível) e **Follow** (*auto-scroll* para a última linha). O *tooltip* no **Follow** explica o comportamento.

## 5.2 Limites de buffer

O console retém até 200 000 caracteres em memória; ao ultrapassar esse limite mais 50 000 de *slack*, o trecho mais antigo é removido em uma operação só. Isso evita o consumo descontrolado de memória quando um sketch *Plotter* envia ≈11 KB/s sem parar.

**Por que não retemos tudo?**

Reter o histórico inteiro fazia o slot `update_uart` virar $O(n^2)$: cada concatenação de `str + str` reconstrói a string toda. Em alguns segundos a UI chegava a megabytes e a thread do Qt travava por completo. O *cap* mantém a janela de tempo útil para depuração (≈17 s a 11 KB/s) sem o custo quadrático.

## 5.3 Envio para o chip

A barra inferior tem um campo de texto e o botão **Send**. Pressionar `Enter` dispara o mesmo comportamento. O texto digitado é enviado para o RX da UART do chip, com um `\n` adicionado ao final — a maioria dos sketches usa `Serial.readStringUntil('\n')` ou similar.

# 6. Plotter

O Plotter é uma janela com uma área de gráfico em *rolling window* mostrando até 14 sinais simultâneos do chip selecionado. Acessível pelo botão **Plot >**.

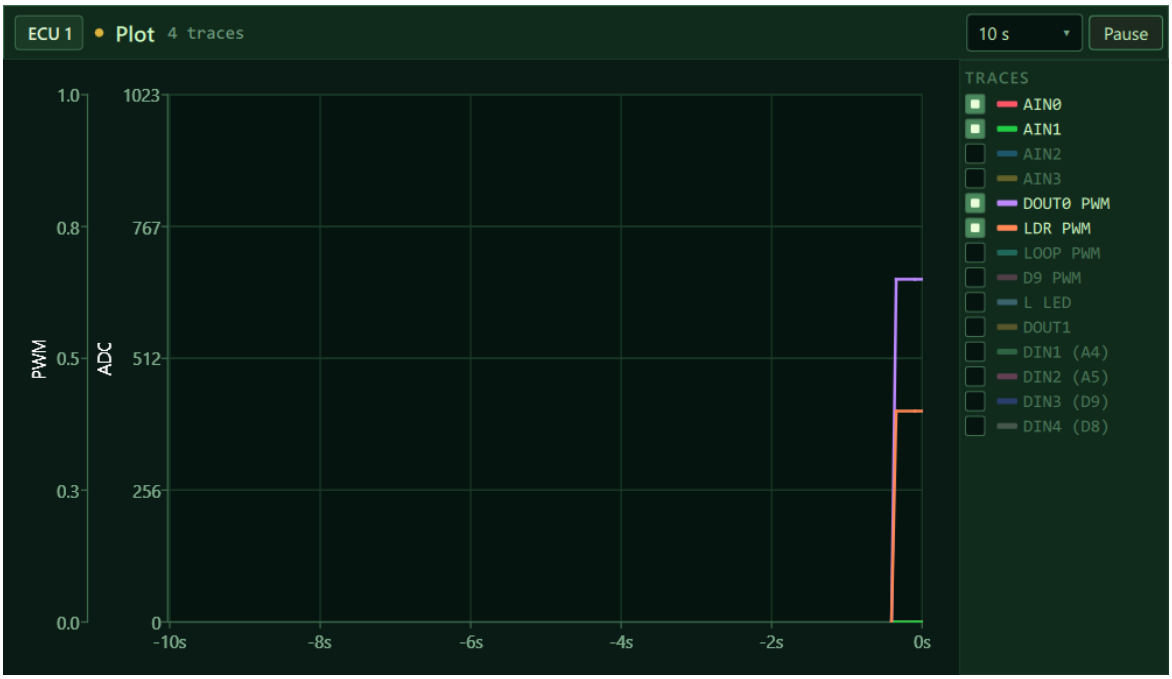


Figura 3: Plotter com legenda dos 14 sinais disponíveis. Eixo Y esquerdo (0–1023) para ADCs, eixo Y direito (0,0–1,0) para canais PWM. Janela temporal selecionável (10 s neste caso) e botão *Pause* no canto superior direito.

## 6.1 Cabeçalho

- **Window** (`10 s` / `30 s` / `1 min` / `5 min`): seletor da janela temporal visível no eixo X. O eixo é "segundos antes de agora"; `x = 0` é o instante atual.
- **Pause/Resume**: congela o gráfico mantendo a coleta de amostras; ao retomar, todos os pontos coletados durante a pausa aparecem.

## 6.2 Catálogo de sinais

A coluna direita lista todos os sinais expostos pelo bridge:

- `adc:0`–`adc:7`: leituras analógicas (eixo Y esquerdo, escala 0–1023);
- `pwm:PD3`, `pwm:PD5`, `pwm:PD6`, `pwm:PB1`: *duty cycles* dos canais PWM (eixo Y direito, escala 0,0–1,0);
- Mais traços digitais conforme o engine os exponha.

Cada *checkbox* liga/desliga um traço; o *swatch* de cor escurece quando o traço está desligado.

## 6.3 Amostragem

O bridge da UI amostra a cada ≈50 ms (20 Hz) o estado de cada sinal e mantém um *ring buffer* de 6 000 amostras por sinal. Para janelas de 5 minutos, isso dá ≈20 amostras/s — suficiente para detectar variações de PWM com frequência ≤10 Hz sem agredir o GPU.

# 7. Gravação de firmware

## 7.1 Pelo botão Flash da UI

O fluxo nativo do aneb-sim usa o botão **Flash >** em cada painel ECU. Ele abre a janela do `avrdude` embutido, carrega o Optiboot empacotado no *engine* e dispara `avrdude -c arduino` via TCP em `net:127.0.0.1:8600+i` (porta de *flasher*).

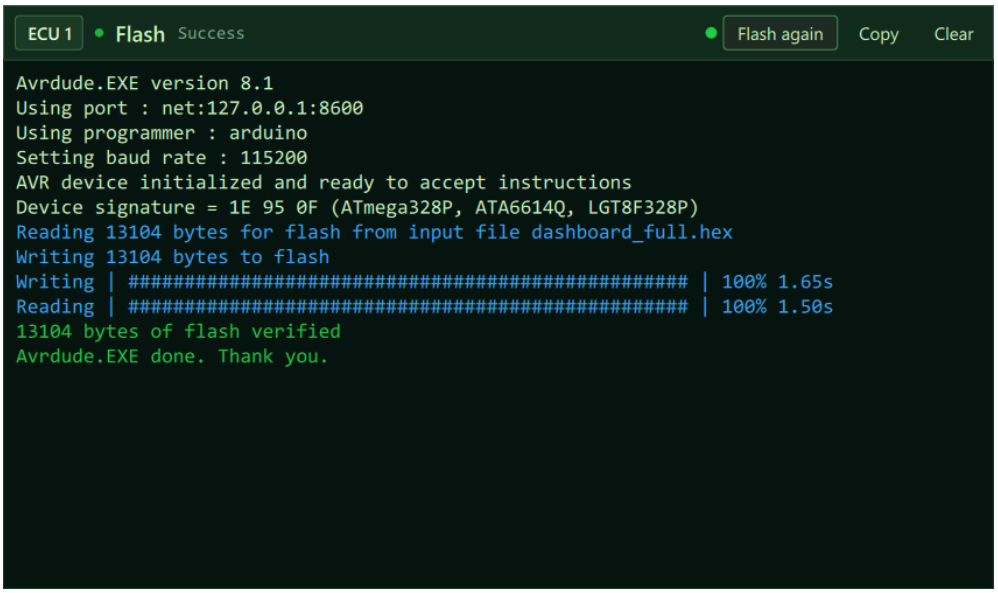


Figura 4: Janela do `avrdude` com log colorizado: linhas informativas em ciano, *Writing/Reading* com barra de progresso, *verified/Avrdude.EXE done* em verde indicando sucesso. O cabeçalho mostra o status atual (*Success*) e três ações (*Flash again / Copy / Clear*).

O cabeçalho mostra o status atual (*Idle / Flashing... / Success / Done*) com cor correspondente, um ponto pulsante enquanto a gravação está em curso, e três ações: **Flash again**, **Copy** (copia o log inteiro para o clipboard) e **Clear**.

## 7.2 Externamente (Remote Flasher, Arduino IDE, linha de comando)

Como o engine deixa o Optiboot pré-carregado em todos os chips, o mesmo `.hex` pode ser gravado por qualquer ferramenta `avrdude`-compatível que aceite o transporte `net:host:port`:

```
avrdude -c arduino -p atmega328p -b 115200 \
        -P net:127.0.0.1:8600 \
        -U flash:w:meu_sketch.ino.hex:i
```

A porta `8600+i` (*flasher port*) tem semântica DTR-pulse: cada `connect` dispara um reset interno do chip, e o `disconnect` dispara um *soft reset* (MCUSR=0) que faz

Optiboot pular direto para o sketch recém-gravado. O usuário não precisa mexer em nenhum botão de *reset*.

**Bridge da UI vs. flasher**

A bridge interna do aneb-sim usa a porta *passiva* `8700+i`; ela é expulsa automaticamente quando um `avrdude` chega na porta `8600+i`. Após ≈5 s do final da gravação, a bridge se reconecta sozinha e a saída UART do firmware volta a fluir para a porta COM virtual — não há configuração nem botão para apertar.

# 8. Integração com o Remote Firmware Flasher

A partir da versão 2.4.0, o **Remote Firmware Flasher** detecta automaticamente o aneb-sim instalado na mesma máquina e adiciona a entrada `Localhost (aneb-sim)` ao combo de PCs. Selecionando essa entrada, todas as abas (**Flash**, **Serial**, **CAN**) operam contra o simulador em vez de uma placa remota.

## 8.1 Fluxo

1. Inicie o aneb-sim com `./scripts/run-ui.sh`.
2. Abra o Remote Flasher.
3. Na aba **Flash**, selecione `Localhost (aneb-sim)` no combo **Computer**, a ECU desejada e o `.hex`.
4. Clique em **Upload + Reset + Flash**. Como o `.hex` já está no disco, o *upload* é pulado; o *reset* usa o pulso DTR pela conexão TCP; o *flash* é `avrdude -c arduino -P net:127.0.0.1:8600+i`.
5. Aguarde ≈5–10 s após o término. A bridge da UI reconecta sozinha; abrir COM11 (ou a COM equivalente da ECU) na aba **Serial** mostra a saída UART do sketch que está rodando.

**Mais detalhes**

O capítulo correspondente do manual do Remote Flasher (`docs/manual.tex`, seção "*Modo local: simulador aneb-sim*") descreve o fluxo do lado do Remote Flasher, com as limitações conhecidas (uma só placa simulada, janela de 5 s sem bridge após cada gravação, nome do `.hex` aparece como "*(uploaded via avrdude)*" no aneb-sim porque o engine não recebe o nome do arquivo via STK500).

# 9. Exemplos de firmware

A pasta `firmware/examples/` traz *sketches* Arduino que exercitam cada periférico da placa em isolamento. Cada um vem acompanhado de `.ino`, `.hex` pré-compilado (com e sem bootloader) e uma *tag* no `README.md` indicando em qual ECU faz mais sentido rodar.

## 9.1 Catálogo

- **blink**: pisca o LED L (D13) a 1 Hz. Sanidade básica.
- **io_test**: *smoke test* que toca em DIN1–4, AIN0–3, DOUT0/1, L e LDR_LED simultaneamente, exibindo o estado das entradas no LCD.
- **lcd_hello**: inicializa o display $I^2C$ 16×2 a 0x27 e exibe duas linhas com texto deslizante.
- **buzzer_tone**: toca um arpejo de C maior na saída BUZZ (PD7) usando `tone()` / `noTone()`.
- **adc_to_pwm**: lê AIN0 e modula DOUT0 (PD3) via PWM de forma linear. Plotador mostra os dois sinais paralelos.
- **button_counter**: contador de eventos com debounce nos botões DIN1–4 e exibição no LCD; DIN1=+1, DIN2=−1, DIN3=zera, DIN4=alterna sinal.
- **serial_echo**: lê linhas da UART e devolve com prefixo numérico. Útil para validar o caminho bridge → COM virtual em ambos os sentidos.
- **ldr_loop** (ECU1): malha óptica fechada — PWM em PD6 acende o LDR_LED e o sketch lê o LDR em A0, gerando uma curva de transferência ótica.
- **loop_feedback** (ECU2): malha elétrica via D5 → A7 com controlador proporcional simples ($K_P = 0,2$). Alvo alterna entre 30 % e 70 % a cada 1,5 s; saída impressa em CSV para o Plotador.
- **can_heartbeat**: quatro variantes (ECU1–ECU4) que transmitem um contador na CAN periodicamente. Demonstra comunicação inter-ECU.
- **can_busoff_demo**: reporta TEC/REC/EFLG da MCP2515 a 1 Hz e se recupera automaticamente de *bus-off*. Use junto com a injeção de erros do menu CAN da UI ou o *scenario player* para ver as transições.
- **mcu_mode_select** (MCU): máquina de estados de 4 estados (*Electric*, *Auto*, *Manual*, *Diag*) controlada pelos seletores D8/D9 do MCU.
- **kitchen_sink**: *todos* os periféricos em um sketch só (DOUT, DIN, AIN, PWM, BUZZ, LCD, CAN). Bom para verificar que tudo está plugado depois de uma reinstalação.

## 9.2 Compilando

```
arduino-cli core install arduino:avr
arduino-cli lib  install "mcp_can"
arduino-cli lib  install "LiquidCrystal I2C"

cd firmware/examples
./build_all.sh
```

O script gera tanto `<nome>.hex` (sem bootloader, para o caminho `cmd_load` da UI) quanto `<nome>_with_bootloader.hex` (com Optiboot embutido, para uso com `avrdude` externo). Como o *engine* do aneb-sim já pré-carrega Optiboot em `chip_init`, qualquer das duas variantes funciona em qualquer caminho de gravação.

## 9.3 Fluxo recomendado de aprendizado

Para uma turma nova:

1. Comece por **io_test** — vai mostrar todos os widgets da UI ao mesmo tempo, dando intimidade com a placa.

2. Cada periférico em isolado: **buzzer_tone**, **adc_to_pwm**, **button_counter**, **lcd_hello**, **serial_echo**.
3. Conceitos de controle: **ldr_loop** (ECU1) e **loop_feedback** (ECU2) introduzem leitura analógica + atuação proporcional.
4. Comunicação: **can_heartbeat** a quatro nós, depois **can_busoff_demo** com injeção de erros para ver bus-off acontecer e ser recuperado.
5. **kitchen_sink** fecha o ciclo — combine seu próprio sketch a partir das partes deste.

# 10. Cenários: *teaching aids* reproduzíveis

A pasta `demos/` contém arquivos `.jsonl` que o engine sabe replantar via cronograma absoluto. São roteiros para aulas: em vez de digitar comandos manualmente, o instrutor seleciona um cenário e os chips passam pelos estados encenados.

## 10.1 Formato

Um objeto JSON por linha. O objeto é um comando normal do engine (ver `docs/PROTOCOL.md`) com um campo extra `at_ms` indicando *quando* (em milissegundos desde o início do script) aplicá-lo:

```
{"v": 1, "at_ms":  500, "c": "force_busoff", "chip": "ecu2"}
{"v": 1, "at_ms": 3000, "c": "can_recover",  "chip": "ecu2"}
```

Linhas começando com `#` ou `//` são comentários e são ignoradas. Linhas em branco também.

## 10.2 Catálogo

- **busoff_recovery.jsonl**: força ECU2 a entrar em *bus-off* duas vezes e se recuperar. Use junto com `can_busoff_demo.hex` flashado em ECU2 para ver as transições no Console.
- **tec_climb.jsonl**: incrementa TEC de ECU3 em 16 unidades a cada 500 ms, atravessando os limiares *active* → *passive* (TEC=128) e *passive* → *bus-off* (TEC=256), depois recupera. Curva fica visível no Plotter.
- **multi_node_arbitration.jsonl**: injeta três quadros CAN com IDs diferentes no mesmo *tick*, ilustrando a ordem de entrega do barramento simulado (em ordem de *attach*, não de ID).

## 10.3 Como executar

**Pela UI**: clique **Toolbar** → **Scenario >** e escolha um item do menu. "Choose .jsonl file..." abre um diálogo para selecionar qualquer arquivo do disco.

**Pela CLI**: passe `--script` ao binário do engine — útil para demonstrações automatizadas e *screencasts*:

```
./build/aneb-sim/aneb-sim.exe \
```

```
    --script=demos/busoff_recovery.jsonl --no-mcu
```

**Por JSON-Lines**: envie um comando `run_script` diretamente para a *stdin* do engine:

```
{"v": 1, "c": "run_script", "path": "demos/busoff_recovery.jsonl"}
```

### 10.4 Criando seus próprios cenários

Use qualquer editor de texto. Recomendações:

- `at_ms` é *absoluto* a partir do início do script. Se duas linhas têm o mesmo `at_ms`, ambas disparam tão próximo quanto o sistema operacional permite (típico em demos de *race* de barramento).
- Se o tempo `at_ms` já passou ao chegarmos na linha (por algum atraso de *scheduling*), a linha dispara imediatamente em vez de *retroativamente*.
- Cenários podem ser executados em paralelo — a fila de comandos é única, então as linhas se intercalam pela ordem cronológica em que ficam prontas.

## 11. Controle programático: protocolo JSON-Lines

O *engine* `aneb-sim.exe` pode ser executado diretamente sem a UI Python — útil para testes automatizados, demos de aula ou *wrappers* customizados.

```
./build/aneb-sim/aneb-sim.exe
```

A *stdin* aceita comandos JSON-Lines, um por linha; a *stdout* emite eventos no mesmo formato. Veja `docs/PROTOCOL.md` para o esquema completo. Exemplos:

```
% Carregar firmware na ECU1:
{"v":1,"c":"load","chip":"ecu1","path":"firmware/examples/blink.hex"}

% Pressionar o botão DIN1 (PC0):
{"v":1,"c":"din","chip":"ecu1","pin":"PC0","val":1}

% Ler a porta TCP UART da ECU1:
%   nc 127.0.0.1 8700      (lado bridge  passivo)
%   nc 127.0.0.1 8600      (lado flasher  dispara reset!)
```

## 12. Solução de problemas

### 12.1 A UI abre mas não vejo dados nas portas COM

1. Verifique que `scripts\setup_com.bat` foi executado como administrador. No **Gerenciador de Dispositivos → Portas (COM e LPT)** devem aparecer os pares (ECU1 (aneb-sim) etc.).

2. No log do `run-ui.sh` você deve ver `bridge connected: ecu1 TCP:8700 <-> COM10`; se a linha não aparece, a bridge falhou ao abrir COM10.
3. Confirme que nenhum outro processo (PuTTY, Arduino IDE em *Serial Monitor*, instância antiga do Remote Flasher) está com a porta COM aberta.
4. Reinicie o aneb-sim. A bridge reconecta automaticamente até sucesso, com *backoff* exponencial.

## 12.2 Avrdude trava em "not in sync"

- O chip provavelmente não tem Optiboot — normalmente isso não acontece (o engine pré-carrega Optiboot em `chip_init`), mas se você descarregou o firmware via *Eject* e tentou gravar antes de o engine voltar, pode ter pegado o stub de *halt*. Solução: use o botão **Flash** da UI uma vez (ela recarrega Optiboot explicitamente) e tente o avrdude externo de novo.
- Confirme que está usando `-P net:127.0.0.1:8600` (e não `8700`, que é a porta passiva da bridge).

## 12.3 O simulador trava ou "Not Responding"

Em geral é um *bottleneck* de evento na thread de GUI:

- Sketches que enviam UART a ≥10 KB/s *sem que a bridge esteja conectada* caíam para o caminho JSON do engine e podiam bombardear o thread Qt; isso é mitigado por um *rate limit* de 100 Hz com *batching* no `sim_loop` desde a v0.9.
- Pinos que oscilam em frequência altíssima (*chip-select* de SPI, por exemplo) são limitados a 100 Hz por par (chip, pino) com de-duplicação de valores idênticos.

Se ainda assim travar, abra um *issue* com a versão do firmware testado — é um candidato a sintonização.

## 12.4 Quero recompilar Optiboot ou usar outro bootloader

O Optiboot é embutido no *engine* a partir de `aneb-sim/firmware/bootloaders/optiboot_atmega328`. Para trocar o bootloader, substitua esse arquivo e refaça `./scripts/build.sh` — o CMake gera `optiboot_data.h` a partir do `.hex` no momento da configuração (com `NO_HEX_CONVERSION` para não interpretar os pares hex como bytes binários).

# Apêndice A — Mapa de portas TCP

| Chip | Porta flasher (avrdude) | Porta bridge (passiva) |
|---|---|---|
| ECU1 | 8600 | 8700 |
| ECU2 | 8601 | 8701 |
| ECU3 | 8602 | 8702 |
| ECU4 | 8603 | 8703 |
| MCU | 8604 | 8704 |

# Apêndice B — Estrutura de diretórios

```
aneb-sim/                    -- engine C (simavr + MCP2515 + CAN bus)
  src/                         cores: chip.c, sim_loop.c, uart_server.c
  firmware/bootloaders/        Optiboot empacotado
aneb-ui/                     -- UI PyQt6
  aneb_ui/qml/                 cenas QML (Board, EcuPanel, Theme.js,
     ...)
  aneb_ui/state.py             store de estado central
  aneb_ui/uart_bridge.py       bridge TCP <-> COM virtual
external/simavr/             -- submódulo simavr (commit fixado)
firmware/                    -- .hex de exemplo (examples + mcu/)
scripts/                     -- bootstrap.sh, build.sh, run-ui.sh,
                                setup_com.bat, screenshots/
docs/                        -- ARCHITECTURE.md, PROTOCOL.md,
                                MCP2515_MODEL.md, manual/manual.tex
build/                       -- saídas do CMake (não-versionado)
.venv/                       -- venv Python (não-versionado)
```